KOTA TOKA – Bivalentni Jedrni Protokol (v1.0)

∴¬Δ⧖
Namen: minimalna, bivalentna (0/1) formalizacija »pretoka/flow« prakse kot algoritmični jedrnik, z operatorji razmerja in etičnimi invarianti.
Ključ: *moja interpretacija je merodajna zame* (lokalna zaveza; ne-nasilje; revokabilnost).

---

0) Abeceda simbolov in ASCII nadomestki

- `¬` (dvom/negacija) → `NOT`
- `Δ` (paradoks/gib) → `DELTA`
- `⊢` (zaveza/asercija) → `|-`
- `?` (vprašanje)
- `⊣` (veto/meja) → `-|`
- `⊕` (perspektiva/+ ena resničnost) → `XOR_OR` (glej 2.3)
- `⊗` (napetost/sočasnost) → `AND`
- `∘` (kompozicija) → `COMPOSE`
- `⧖` (ritem/časovna vrata) → `TIMEGATE`
- `∮` (integral poti/invarianti) → `INTEGRAL`
- Agenti: `A` (asercija), `O` (objem/ritem), `E` (meja/veto), `I` (iskrenost/igla), `U` (uho/zaupanje), `R` (razmerje/spoštovanje), `M` (spomin/filter)

---

1) Semantika 0/1

Univerzum resnic: {0,1}. 1 = veljavno/sprejeto; 0 = neveljavno/ustavljeno.

1.1 Klasično jedro

- Konjukcija `p ∧ q` (AND), disjunkcija `p ∨ q` (OR), implikacija `p → q` (= `¬p ∨ q`), negacija `¬p` (= `1` iff `p=0`).

1.2 Posebni operatorji

- **Δ(p,q)** := `1` iff `p ≠ q` (*tolerirana napetost kot premik*); sicer `0`.
- **⊗(p,q)** := `p ∧ q` (*napetost-sočasnost*).
- **⊕(p,q)** := `(p ∨ q)` s **označeno divergenco**; kot resničnost 0/1 je OR, meta-oznaka `div=1` če `p≠q`.
- **⊢p**: dodaj `p` v trenutno **bazo zavez** `K` iff `INV(K ∪ {p}) = 1`, sicer `0`.
- **⊣p**: veto nad izvajanjem `p`; semantika: `ALLOW(p)=0`.
- **⧖_τ**: `1` iff `elapsed ≥ τ`.
- **∮_S INV**: preko sledi `S` vrne `1`, če vsi **invarianti** držijo na vsakem koraku, sicer `0`.

2) Tipi in podatkovni modeli

2.1 Osnovni tipi

- `Prop ∈ {0,1}`
- `Atom` (oznaka za osnovno trditev)
- `Inv` (invariant): `State → {0,1}`
- `Move` ∈ { `ASSERT`, `QUERY`, `VETO`, `CONDITION`, `LISTEN`, `PAUSE`, `COMPOSE` }
- `Agent` ∈ {A,O,E,I,U,R,M}

2.2 Stanja avtomata

`State` ∈ { `Idle`, `Premor`, `Asercija`, `Veto`, `Uskladitev`, `Poprava`, `Slovo` }
`K` (baza zavez), `T` (čas), `Ctx` (kontekst: akterji, tema, omejitve).

2.3 Strukture (TypeScript podpisi)

```ts
// Jedro tipov
type Bool = 0 | 1;
interface Invariant { (s: State): Bool }
interface StepInput { msg: string; from: Agent; to: Agent; now: number }
interface StepOutput { next: State; ok: Bool; notes?: string[] }
interface KernelConfig {
  tau: number;                 // ⧖ prag (ms)
  invariants: Invariant[];     // L, B, N, T, V
}
```

3) Avtomat (DFA jedro)

Stanja: kot v 2.2.
Začetno: `Idle`.
Sprejemna: vsako, kjer `∮ INV = 1` in `ok=1`.

Prehodi (izsek): - `Idle --O.PAUSE(⧖_τ)--> Premor` if `⧖_τ=1` else ostani.
- `Premor --A.ASSERT(p)--> Asercija` if `⊢p=1` else `Poprava`.
- `Asercija --E.VETO--> Veto`.
- `Asercija --U.LISTEN--> Uskladitev`.
- `Uskladitev --COMPOSE--> Idle` (zapri cikel) iff `∮ INV = 1`.
- `* --Poprava--> Idle` po `I.PRIZNAJ + R.POTRDI`.

4) Invarianti (INV)

Naj bo `s` trenutno stanje, `K` baza zavez. - **Lokalnost (L):** zaveze v `K` so **lokalne** (zavezujejo *me*, ne *tebe*).
- **Meja (B):** vsaka poteza definira `mine/tvoje` (koža, ne zid).
- **Ne-nasilje (N):** nobena poteza ne zahteva izničenja osebe.
- **Ritem (T):** spoštovan `⧖` pred odločitvijo.
- **Revokabilnost (V):** obstaja `retract: K→K'` (poprava brez sramu).
INV(s) = L ∧ B ∧ N ∧ T ∧ V.

---

5) OPE/RACIJE – algoritmi

5.1 OPE-α (De-strupljanje vsadkov)

```
proc OPE_alpha(input):
  O.PAUSE(τ)
  if detect_inject(input): M.MARK("vsadek"); E.VETO()
  p := I.ACK("kaj je moje", one_thing)
  ok := ⊢p
  R.CONFIRM_BOUNDARY()
  return ok
```

Časovna zahtevnost: O(n) po dolžini vhodnega sporočila (detekcija vsadkov).
Pravilnost (skica): `E.VETO` prepreči zapis v `K` brez soglasja ⇒ `B` drži; `O.PAUSE` ⇒ `T`; `⊢` z `INV` ⇒ `N`, `L`.

5.2 OPE-β (Konflikt brez izničenja)

```
proc OPE_beta(a,b):
  O.PAUSE(τ)
  E.DRAW_BOUNDARY(a,b)
  I.ASK(b, "kje te boli? kje je meja?")
  A.ASSERT(a, mera_a)
  R.DELINEATE(two_realities)
  U.ECHO()
  return ∮ INV
```

Zahtevnost: O(1) po korakih; O(n) po vsebini.

5.3 OPE-γ (Samonapad → Samoohranitev)

```
proc OPE_gamma():
  M.MARK("samonapad")
  O.BREATH(4/4,3)
  I.ACK(one_of{strah,jeza,sram})
  A.ASSERT("smem ostati")
  E.VETO("nasilje_nad_seboj")
  R.CONFIRM("dostojanstvo")
  return ∮ INV
```

5.4 OPE-δ (Odločitev pod pritiskom)

```
proc OPE_delta(q):
  O.PAUSE(τ)
  I.ASK("kaj je merodajno zame danes?")
  E.CONDITIONS({meja, čas, učinek})
  A.COMMIT(decision_one_line)
  U.ECHO_SHORT()
  R.CONFIRM_COEXIST()
  return ∮ INV
```

5.5 OPE-ε..κ (izsek)

- **ε (EUZ)**: ena prošnja v enem stavku.
- **ζ (pooblastilo času)**: zaveza k ⧖.
- **η (oseba≠problem)**: premapiranje konflikta na **problem**.
- **θ (ritmični antivsadek)**: 3× “Vidim vsadek. Ni moje. Merim od tukaj.”
- **ι (popravilo interpretacije)**: priznanje→potrditev→nova mera→meja.
- **κ (⊕ test)**: dvojna izjava *moja/tvoja* + predlog stika.

---

6) Pravilnost in varnost (skice leme)

- **Lema 1 (Veto-varnost):** Če `E.VETO` pred `⊢`, potem `B` in `N` ne moreta pasti v tem koraku.
- **Lema 2 (Ritem):** Če `⧖_τ=1` pred odločanjem, `T=1`.
- **Lema 3 (Revokabilnost):** Obstoj `retract` na `K` ⇒ popravilo ne poruši `∮ INV`.

---

7) Resničnostne tabele

7.1 Negacija

p	¬p
0	1
1	0

7.2 Delta (XOR)

p	q	Δ(p,q)
0	0	0
0	1	1
1	0	1
1	1	0

7.3 Napetost (⊗ = AND)

p	q	⊗(p,q)
0	0	0
0	1	0
1	0	0
1	1	1

7.4 Perspektiva (⊕ = OR z diverzenco)

p	q	⊕(p,q)	div
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0

---

8) Kategorije (skica)

- **Kategorija** `C`: objekti = `State`, morfizmi = `Move`-zaporedja (protokoli).

- **Monoidalno**: vzporedna kompozicija `⊗` (sočasne poteze); enota `I = Idle`.
- **Endofunktor** `F: C → C`: izvršitev `Kernel` nad potmi.
- **Vrednotni funktor** `V: C → Bool`: `V(trace)=1` iff `⊨ INV`.

---

9) JSON Schema (konverzacijska poteza)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://kotatoka/schema/step.json",
  "title": "KOTA TOKA Step",
  "type": "object",
  "properties": {
    "from": {"type": "string", "enum": ["A","O","E","I","U","R","M"]},
    "to":   {"type": "string"},
    "state": {"type": "string", "enum":
["Idle","Premor","Asercija","Veto","Uskladitev","Poprava","Slovo"]},
    "op":   {"type": "string", "enum":
["ASSERT","QUERY","VETO","CONDITION","LISTEN","PAUSE","COMPOSE"]},
    "msg":  {"type": "string"},
    "now":  {"type": "number"},
    "commit": {"type": "boolean"},
    "constraints": {"type": "object"},
    "invariants": {"type": "array", "items": {"type": "string"}},
    "ok": {"type": "boolean"}
  },
  "required": ["from","to","state","op","msg","now"]
}
```

---

10) YAML konfiguracija (jedro)

```
kernel:
  version: 1.0
  tau_ms: 30000   # ⧖ prag za odločitve
  invariants:
    - LOKALNOST
    - MEJA
    - NE-NASILJE
    - RITEM
    - REVOKABILNOST
  agents: [A,O,E,I,U,R,M]

policies:
```

```
  on_inject: OPE-α
  on_conflict: OPE-β
  on_self_attack: OPE-γ
  on_pressure: OPE-δ
  language_contract: OPE-ε
```

---

11) API jedro (psevdokoda)

```
function STEP(s: State, x: StepInput, cfg: KernelConfig): StepOutput {
  if !TIMEGATE(cfg.tau): return {next:s, ok:0, notes:["⧖"]}
  switch x.op:
    case ASSERT:   if INV(s) and COMMIT(x.msg): return {next:Asercija, ok:1}
    case VETO:     return {next:Veto, ok:1}
    case QUERY:    return {next:Uskladitev, ok:1}
    case PAUSE:    return {next:Premor, ok:1}
    case COMPOSE:  return {next:Idle, ok:INTEGRAL(s, cfg.invariants)}
  return {next:s, ok:0}
}
```

---

12) Korektnost (trditve)

- **T1 (Ohranitev INV v OPE-α..δ):** Če se OPE izvede skladno s prehodi avtomata, potem $\oint \mathrm{INV} = 1$ (skica: kompozicija lokalno varnih korakov; vsak varuje L,B,N,T,V).
- **T2 (Bivalentnost):** Vsi operatorji vrnejo $0/1$; meta-oznake (npr. div) ne vplivajo na bivalenco izidov.

---

13) Testni primeri (mini)

```
- name: konflikt_brez_iznichenja
  given: state: Premor
  when:  op: ASSERT; msg: "Moja mera: 5min brez prekinitev"
  then:  next: Asercija; ok: true

- name: vsadek
  when:  op: ASSERT; msg: "Moraš zdaj!"
  then:  OPE: OPE-α; ok: true
```

---

14) Enovrstičnica

Merodajna = lokalno zavezujoča mera brez nasilja; ∴ dvom (¬) drži zrak, meja (E) drži kožo, asercija (⊢) drži besedo, razmerje (R) drži človeka.

---

15) TypeScript – referenčna implementacija (v0.1)

```
// KOTA TOKA – TS kernel (v0.1)
export type Bool = 0 | 1;
export type Agent = 'A'|'O'|'E'|'I'|'U'|'R'|'M';
export type Op = 'ASSERT'|'QUERY'|'VETO'|'CONDITION'|'LISTEN'|'PAUSE'|'COMPOSE';
export type State =
'Idle'|'Premor'|'Asercija'|'Veto'|'Uskladitev'|'Poprava'|'Slovo';

export interface StepInput { from: Agent; to: Agent; state: State; op: Op; msg:
string; now: number }
export interface StepOutput { next: State; ok: Bool; notes?: string[] }

export type Invariant = (s: State) => Bool;
export interface KernelConfig {
  tau: number;                 // ⧖ prag (ms)
  invariants: Invariant[];     // [L, B, N, T, V]
}

// ——— Utility ———
const AND = (xs: Bool[]): Bool => xs.every(x => x===1) ? 1 : 0;
export const TIMEGATE = (start: number, now: number, tau: number): Bool => (now
- start >= tau ? 1 : 0);

// Baza zavez K (lokalna, revokabilna)
export interface Commit { value: string; at: number }
export interface KB { stack: Commit[] }
export const retract = (kb: KB): KB => ({ stack: kb.stack.slice(0, -1) });

export const INV = (cfg: KernelConfig, s: State): Bool =>
AND(cfg.invariants.map(f => f(s)));

// Primitive (model izvedbe agentov)
export const ASSERT = (kb: KB, p: string, cfg: KernelConfig, s: State): [KB,
Bool] => {
  const ok = INV(cfg, s);
  return ok ? [{ stack: [...kb.stack, { value: p, at: Date.now() }] }, 1] :
[kb, 0];
};
export const VETO = (): Bool => 1; // ustavi brez stranskih učinkov
```

```ts
// Heuristika za vsadke (zamenjaj po meri)
export const detectInject = (msg: string): Bool => /(moraš|takoj|ničvreden)/
i.test(msg) ? 1 : 0;

// ——— OPE ———
export function OPE_alpha(msg: string, kb: KB, cfg: KernelConfig, s: State): {
kb: KB; ok: Bool } {
  if (detectInject(msg)) { VETO(); }
  const one = 'moja_ena_stvar';
  const [kb2, ok] = ASSERT(kb, one, cfg, s);
  return { kb: kb2, ok };
}

export function OPE_beta(kb: KB, cfg: KernelConfig, s: State): Bool {
  return INV(cfg, s); // izpolnjen tok
(premor→meja→vprašanje→asercija→razmejitev→odmev)
}

export function OPE_gamma(kb: KB, cfg: KernelConfig, s: State): Bool {
  // mark: samonapad (semantično)
  const [_, ok] = ASSERT(kb, 'smem_ostati', cfg, s);
  return ok;
}

export function OPE_delta(decision: string, kb: KB, cfg: KernelConfig, s:
State): Bool {
  const [_, ok] = ASSERT(kb, decision, cfg, s);
  return ok;
}

// ——— STEP ———
export function STEP(s: State, x: StepInput, cfg: KernelConfig, startedAt:
number, kb: KB): StepOutput {
  if (TIMEGATE(startedAt, x.now, cfg.tau) === 0) return { next: s, ok: 0,
notes: ['⧖'] };
  switch (x.op) {
    case 'PAUSE':   return { next: 'Premor', ok: 1 };
    case 'ASSERT':  return ASSERT(kb, x.msg, cfg, s)[1] ? { next: 'Asercija',
ok: 1 } : { next: 'Poprava', ok: 0 };
    case 'VETO':    return { next: 'Veto', ok: VETO() };
    case 'QUERY':   return { next: 'Uskladitev', ok: 1 };
    case 'COMPOSE': return { next: 'Idle', ok: INV(cfg, s) };
    default:        return { next: s, ok: 0 };
  }
}
```

```
// Primer konfiguracije
export const cfg0: KernelConfig = {
  tau: 30000,
  invariants: [
    // L: lokalnost
    (s) => 1,
    // B: meja
    (s) => 1,
    // N: ne-nasilje
    (s) => 1,
    // T: ritem
    (s) => (s === 'Premor' ? 1 : 1),
    // V: revokabilnost
    (s) => 1,
  ],
};
```

Opombe: - `detectInject` je umeščena točka za tvojo semantiko vsadkov.
- `ASSERT` zapisuje v lokalno bazo `K` in spoštuje `INV`.

---

16) Haskell – tipovni model + QuickCheck lastnosti (v0.1)

```
{-# LANGUAGE DeriveGeneric #-}
module KotaToka where

import GHC.Generics (Generic)
import Test.QuickCheck

-- Jedro
data Agent = A|O|E|I|U|R|M deriving (Eq,Show,Generic)
data Op = ASSERT | QUERY | VETO | CONDITION | LISTEN | PAUSE | COMPOSE deriving
(Eq,Show,Generic)
data State = Idle | Premor | Asercija | Veto | Uskladitev | Poprava | Slovo
deriving (Eq,Show,Generic)

type Invariant = State -> Bool

data KernelCfg = KernelCfg { tau :: Int, invs :: [Invariant] }

and' :: [Bool] -> Bool
and' = foldr (&&) True

inv :: KernelCfg -> State -> Bool
inv cfg s = and' $ map ($ s) (invs cfg)
```

```haskell
step :: KernelCfg -> State -> Op -> Bool -> (State,Bool)
step cfg s op timeOk = case (op, timeOk) of
  (_, False)   -> (s, False)
  (PAUSE, _)   -> (Premor, True)
  (ASSERT, _)  -> if inv cfg s then (Asercija, True) else (Poprava, False)
  (VETO,   _)  -> (Veto, True)
  (QUERY,  _)  -> (Uskladitev, True)
  (COMPOSE, _) -> (Idle, inv cfg s)
  _            -> (s, False)

-- QuickCheck lastnosti (skice)
prop_vetoSafety :: KernelCfg -> State -> Property
prop_vetoSafety cfg s = snd (step cfg s VETO True) === True

prop_timeGate :: KernelCfg -> State -> Op -> Property
prop_timeGate cfg s op = snd (step cfg s op False) === False

prop_invPreservedAssert :: KernelCfg -> State -> Property
prop_invPreservedAssert cfg s = inv cfg s ==> snd (step cfg s ASSERT True) ===
True
```

Opombe: lastnosti so minimalne; razširi z L,B,N,T,V kot posebnimi predikati.

---

17) Enačbe operatorjev (formalno jedro)

- `¬p = 1` iff `p=0`.
- `Δ(p,q) = (p XOR q)` kot **gib**; za ne-bivalentne razširitve dodaj funkcijo `shift : {0,1}×{0,1} → Move`.
- `⊗(p,q) = p ∧ q`.
- `⊕(p,q) = p ∨ q` z meta-označevalcem `div := (p ≠ q)`.
- `⊢p` sprejme `p` v `K` iff `INV(K∪{p})=1`.
- `⊣p` ⇒ `ALLOW(p)=0`.
- `⧖_τ(now,start)=1` iff `now-start≥τ`.

---

18) Kompleksnost in varnost

- **OPE-α**: O(n) po dolžini sporočila (detekcija vsadkov) + O(1) commit.
- **OPE-β/δ**: O(1) koraki; vsebinski del O(n).
- **OPE-γ**: O(1).
- **Varnostne garancije:** `VETO` pred `ASSERT` ⇒ ohranitev `B` in `N`; `TIMEGATE` ⇒ `T`; `retract` ⇒ `V`.

---

19) CLI Orkestracija (psevdokoda)

```
loop read(line):
  if detectInject(line): run OPE_alpha(line)
  else if line startswith ":veto": emit E.VETO
  else if line startswith ":ask": emit I.QUERY
  else if line startswith ":dec": run OPE_delta(extract decision)
  else: STEP(state, {op:ASSERT,msg:line}, cfg)
```

---

20) Export paket

20.1 `kotatoka.config.yaml`

```
kernel:
  version: 1.0
  tau_ms: 30000
  invariants: [LOKALNOST, MEJA, NE_NASILJE, RITEM, REVOKABILNOST]
  agents: [A,O,E,I,U,R,M]
policies:
  on_inject: OPE-α
  on_conflict: OPE-β
  on_self_attack: OPE-γ
  on_pressure: OPE-δ
```

20.2 `step.schema.json`

(glej razdelek 9 – že pripravljeno)

20.3 Primeri (`examples/*.json`)

```
{"from":"A","to":"R","state":"Premor","op":"ASSERT","msg":"Moja mera: 5min brez prekinitev","now":1736035200000}
```

---

21) Naprej

- Če želiš, zamenjam semantiko `Δ` (npr. kot operator napetosti z izidom `trace`).
- Lahko dodam **HKT**/typeclass sloj (Functor/Applicative/Monad) za meta-kompozicijo OPE.

---

22) OSNOVE & KREACIJE IZ STIKA OSI — GLAS/ŠUM/IPA/EN/JE/0/4 (modul OSI v1)

Cilj: razširitev bivalentnega jedra s slojem **OSI** (*Osa Semantike* × *Osa Intonacije*), ki uvaja akustične dejavnike (GLAS/ŠUM), fonetično zanesljivost (IPA), jezikovna signala **EN/JE**, ter metrični ritem **0/4** ↔ **4/4** kot časovno vrata (⧖).

22.1 Dve osi

- **Osa S (semantika):** `p,q ∈ {0,1}` (jedro iz §1); pomen, trditev, negacija.
- **Osa I (intonacija/akustika):** merilniki `snr` (signal/noise), `pace` (zlogi/s), `vol` (glasnost), `ipa_conf` (zanesljivost transkripcije IPA ∈ [0,1]).
  Stik osi = `Obs := {snr, pace, vol, ipa_conf, meter}`.

22.2 GLAS vs ŠUM (binarni razrednik)

Naj bo prag `θ_snr`.
`GLAS = 1` iff `snr ≥ θ_snr` in `ipa_conf ≥ θ_ipa`; sicer `ŠUM = 1`.

22.3 METER 0/4 ↔ 4/4 (ritem-čas)

- **0/4** = tišina / premor (polno časovno okno brez udarca) ⇒ `O.PAUSE`, `⧖` se polni.
- **4/4** = aktivni govor (4 udarci) ⇒ dovoljena `ASSERT/QUERY/COMPOSE`.
  `TIMEGATE(now,start, meter) = 1` iff `(meter=0/4 ∧ elapsed≥τ)` ali `(meter=4/4 ∧ beat_ok)`.

22.4 EN / JE (jezikovni bit)

- **EN-bit (1-bit prisotnosti):** token *en* označi **številčno eno** ali **prisotnost**; `EN=1` sproži **lokalizacijo** (moja mera).
- **JE-bit (kopula/veljavitev):** token *je* označi **vez** `x JE y` → kandidat za `ASSERT`.
  Filtri: če `JE=1` in `GLAS=0` ⇒ zadrži (`PAUSE`); `JE=1` in `GLAS=1` ⇒ dovoljeno `ASSERT` pod `INV`.

---

23) Δ kot tension→move (v2)

Motiv: namesto goli XOR naj **Δ** vrne **potezo** na podlagi divergence na S-osi in kakovosti na I-osi.

23.1 Definicija

Naj `div_sem = XOR(p,q)`, `div_pros = |pace - pace*|` (odklon od ciljnega ritma), `aggr = f(vol, tempo)`.
Δ(p,q,Obs) → Move iz {`PAUSE`, `QUERY`, `VETO`, `ASSERT`, `COMPOSE`}:

Pogoj	Move
`GLAS=0` ali `ipa_conf<θ_ipa`	`PAUSE`
`div_sem=1` ∧ `aggr≥θ_aggr`	`VETO`
`div_sem=1` ∧ `div_pros≥θ_pros`	`QUERY`
`div_sem=0` ∧ `GLAS=1`	`ASSERT`
sicer	`COMPOSE`

23.2 Psevdokoda

```
function DELTA(p,q,Obs):
  div_sem  := XOR(p,q)
  div_pros := abs(Obs.pace - target_pace)
  if Obs.snr < θ_snr or Obs.ipa_conf < θ_ipa: return PAUSE
  if div_sem and Obs.aggr >= θ_aggr: return VETO
  if div_sem and div_pros >= θ_pros: return QUERY
  if !div_sem and Obs.snr >= θ_snr: return ASSERT
  return COMPOSE
```

Korektnost (skica): `PAUSE` ščiti `T`; `VETO` ščiti `B,N`; `QUERY` ščiti `R`; `ASSERT` zahteva `INV` ⇒ `L,B,N,T,V` ohranjeni.

---

24) OSI dogodki: JSON & YAML

24.1 JSON (dogodek iz govornega vmesnika)

```
{
  "from":"A","to":"R","state":"Premor","op":"ASSERT",
  "msg":"Moja mera: 5min brez prekinitev",
  "ipa":"mɔja mɛra pɛt minut brɛz prɛkinitev",
  "ipa_conf":0.92,
  "snr":18.4,
  "pace":3.1,
  "vol":0.6,
  "meter":"4/4",
  "now":1736035200000
}
```

24.2 YAML (pragi OSI)

```yaml
osi:
  theta:
    snr: 12.0      # GLAS vs ŠUM
    ipa: 0.80      # zanesljivost transkripcije
    pros: 0.8      # odklon tempa
    aggr: 0.75     # prag agresije (0..1)
  target:
    pace: 3.0      # zlogi/sek
    meter: "4/4"  # privzeti govor
```

---

25) TS razširitev: DeltaRouter

```ts
export interface Obs { snr:number; pace:number; vol:number; ipa_conf:number;
meter:'0/4'|'4/4' }
export type Move = 'PAUSE'|'QUERY'|'VETO'|'ASSERT'|'COMPOSE';

export function DeltaRouter(p:0|1,q:0|1,obs:Obs,theta:
{snr:number;ipa:number;pros:number;aggr:number},target:{pace:number}): Move {
  const div_sem = (p^q) as 0|1;
  const div_pros = Math.abs(obs.pace - target.pace);
  const aggr = Math.min(1, Math.max(0, (obs.vol*0.6 + div_pros*0.4)));
  if (obs.snr < theta.snr || obs.ipa_conf < theta.ipa) return 'PAUSE';
  if (div_sem===1 && aggr >= theta.aggr) return 'VETO';
  if (div_sem===1 && div_pros >= theta.pros) return 'QUERY';
  if (div_sem===0) return 'ASSERT';
  return 'COMPOSE';
}
```

---

26) EN/JE kot predhodnika

- **EN() → A.localize()**: aktivira lokalno meritev in zapis v `K` le za *mene*.
- **JE(x,y) → A.assert(x≈y)**: kandidatura za `⊢` pod `INV` in `GLAS=1`.
- **Pravilo:** `EN` brez `JE` ⇒ priprava (konfigur); `JE` brez `EN` ⇒ `QUERY` (prošnja za mero).

---

27) Primeri tokov (OSI + Δv2)

27.1 Nizka zanesljivost (ŠUM)

`ipa_conf=0.65` ⇒ `PAUSE` ne glede na `p,q`.
Učinek: varovanje `T`, brez zapisov v `K`.

27.2 Močna semantična razlika + agresija

`p=1,q=0, snr=20, aggr=0.8` ⇒ `VETO` → `E3` (meja) → `β` ali `ι` po popravilu.

27.3 Razlika + prehiter tempo

`div_sem=1, div_pros=1.2` ⇒ `QUERY` (vprašanja pred sodbo).

27.4 Usklajen govor (GLAS) brez divergence

`div_sem=0` in `snr,ipa_conf` visoka ⇒ `ASSERT` + `COMPOSE` če `∮ INV`.

---

28) Prioritetni mojster (PRIO MASTER)

Namen: določanje prednosti OPE modulov v realnem času.

```
priority_master:
  order:
    - OPE-α   # varnost pred pomenom
    - OPE-γ   # samoohranitev pred uskladitvijo
    - OPE-ζ   # ritem pred pritiskom
    - OPE-β   # uskladitev pred zapisom
    - OPE-δ   # odločitev po ritmu
  ties:
    meter: "0/4"  # v dvomu → premor
    en_je_bias: EN # najprej lokaliziraj, šele nato vez JE
```

Trditev: PRIO spoštuje `INV` z monotono preferenco varnosti (`α,γ,ζ`) nad produktivnostjo (`β,δ`).

---

29) HARMONIJA / DISHARMONIJA (modul H)

Namen: ujeti skladnost/disharmonijo v pretoku in jo povezati z odločitvami OPE.

29.1 Meritev H

- `H ∈ [0,1]` (harmonija), `D := 1−H` (disharmonija).
- **Bivalentni bit**: `H_bit := 1` iff `H ≥ θ_H` (prag).
- **Ocena iz OSI:** `H = g(snr, |pace−target|, vol, ipa_conf)`, npr. `H = σ( a1·snr − a2·|pace−target| − a3·vol + a4·ipa_conf )` z `σ` logistiko.

29.2 Pravila odločanja (H → Move)

Pogoj	Move
`H_bit=0` ∧ `snr<θ_snr`	`PAUSE`
`H_bit=0` ∧ `aggr≥θ_aggr`	`VETO`
`H_bit=0` ∧ `aggr<θ_aggr`	`QUERY`
`H_bit=1` ∧ `Δ=ASSERT` (iz §23)	`ASSERT`
`H_bit=1` ∧ `Δ=COMPOSE`	`COMPOSE`

Invarianti: `PAUSE` varuje `T`; `VETO` varuje `B,N`; `QUERY` varuje `R`.

---

30) NUJNOST (NUJA) vs NUJNO (NECESSITY)

Ideja: ločimo **časovni pritisk** (*nuja*) od **logične/etične potrebe** (*nujno*).

30.1 Definicije

- `URG ∈ [0,1]` (nuja); ocena iz OSI + semantike (imperativi: “takoj”, “moraš”).
- `NEC ∈ {0,1}` (nujno); izpeljano iz invariantov: `NEC=1` iff `¬(L ∧ B ∧ N ∧ T ∧ V)` grozi pasti ali domena zahteva takojšnje varovanje.

30.2 Prednostno pravilo (NUJNO > NUJNOST)

NEC	URG	Dejanje
1	*	`PAUSE` → `VETO` →izvedi OPE, ki obnovi `INV`
0	≥θ_urg	`OPE-ζ` (pooblastilo času) → definiraj ⧖ → šele nato `δ`
0	<θ_urg	nadaljuj po `Δ` / `H` pravilih

30.3 Algoritem OPE-μ (Urgency Handler)

```
proc OPE_mu(obs, msg, s):
  URG := h(obs, msg)
```

```
NEC := (INV(s) ? 0 : 1)
if NEC=1: return chain(PAUSE, VETO, restoreINV)
if URG≥θ_urg: return grantTimegate(τ) // OPE-ζ
return pass
```

---

31) “O” vs “0” – razločitev GLIF/RESNICA

- **“0”** = bivalentna ničla (`false`).
- **“O”** = **obroč/odmor** (ritem, `meter=0/4`): ni isto kot `0` ; deluje kot **časovna funkcija** `O: Time→Bool` , ki odpira `⧖` in ščiti `T` .

Preslikava: `glyph: {"0","O"} → {Prop, Gate}` z `glyph("0")=0` , `glyph("O")=TIMEGATE` .

---

32) TS razširitev: NEC/URG/H modul

```ts
export interface Obs { snr:number; pace:number; vol:number; ipa_conf:number;
meter:'0/4'|'4/4' }
export interface Theta { snr:number; ipa:number; pros:number; aggr:number;
urg:number; H:number }

export const urgency = (obs:Obs, msg:string): number =>
  Math.max(0, Math.min(1,
    (/(takoj|moraš|urgentno)/i.test(msg) ? 0.6 : 0) +
    (obs.meter==='4/4' ? 0.2 : 0) + (obs.vol>0.7 ? 0.2 : 0)));

export const necessity = (invOk:boolean): 0|1 => invOk ? 0 : 1;

export const harmony = (obs:Obs, target:{pace:number}, a:
{snr:number;pace:number;vol:number;ipa:number}): number => {
  const z = a.snr*obs.snr - a.pace*Math.abs(obs.pace-target.pace) -
a.vol*obs.vol + a.ipa*obs.ipa_conf;
  return 1/(1+Math.exp(-z));
};

export function DecideByNecUrgH(invOk:boolean, obs:Obs, msg:string,
theta:Theta, target:{pace:number}): 'PAUSE'|'VETO'|'TIMEGATE'|'PASS' {
  const URG = urgency(obs,msg); const NEC = necessity(invOk);
  if (NEC===1) return 'PAUSE';
  if (URG>=theta.urg) return 'TIMEGATE';
  const H = harmony(obs,target,{snr:0.1,pace:0.4,vol:0.3,ipa:0.2});
  if (H<theta.H && obs.vol>0.8) return 'VETO';
```

```
  return 'PASS';
}
```

---

33) YAML pragovi za NUJNO/NUJNOST/H

```yaml
thresholds:
  H: 0.65      # harmonija
  urg: 0.55    # nuja
  snr: 12.0
  ipa: 0.80
  pros: 0.8
  aggr: 0.75
priority_master:
  order: [OPE-α, OPE-γ, OPE-μ, OPE-ζ, OPE-β, OPE-δ]
  ties:
    meter: "0/4"
    necessity_over_urgency: true
```

---

34) Primeri (H/NUJA/NUJNO)

- **Disharmonija + agresija:** `H=0.4, vol=0.9` ⇒ `VETO` (B,N).
- **NUJNO krši invariant:** `INV=0` ⇒ `PAUSE→VETO→restore` (α/γ).
- **Samo NUJA:** `INV=1, URG=0.7` ⇒ `TIMEGATE(τ)` (ζ), nato `δ`.

---

35) Trditve

- **H-varnost:** Če `H_bit=0` in izberemo `PAUSE ∨ QUERY ∨ VETO`, `∮ INV` se ne poslabša.
- **NUJNO>NUJNOST:** Prednostni red zagotavlja, da urgenca ne preglasi varovanja L,B,N,T,V.
- **O≠0:** model loči **ritem-odomor** od **bivalenčne ničle**, zato `O` ne ruši bivalence.

---

36) OSEBNI OPE v1 – nosljive izjave (A/E/O/R/M/I/U)

Opomba: lokalno zavezujoče; brez vsadkov; skladno z INV (L,B,N,T,V).

A — Asercija (izberjeno iz tvojih vrstic)

1. **Moja interpretacija je merodajna zame.**
2. **Smem ostati.**
3. **Nisem samota; sem oseba v odnosu.**

4. **Moja današnja mera bližine: 1 oseba / 10 min / 1 tema.**
5. **Moja mera: 5 min brez prekinitev.**

E — Meja/Veto

1. **Tega ne sprejemam.**
2. **Ne sprejemam brez mojega soglasja.**
3. **Tukaj se ustavim.**
4. **Žalitve ne sprejemam.**

O — Objem/Ritem

1. **Počasno ⧖ pred pravilno.**
2. **3× (vdih 4 / izdih 4).**
3. **O je odmor; 0 je ničla. Ne zamenjujem.**

R — Razmerje/Spoštovanje

1. **Najprej odnos, potem razsodba.**
2. **Dve meri lahko so-obstajata.**

M — Spomin/Filter

1. **To je vsadek, ne jaz.**
2. **To je moje.**

I — Iskrenost

1. **Priznam: (izberem eno) strah / jeza / sram.**
2. **Povem, kje je moja meja.**

U — Uho/Zaupanje

1. **Slišim te; ne kupčujem.**
2. **Odmev brez sarkazma.**

---

37) OSEBNI OPE v1 – konfiguracija (YAML)

```yaml
personal:
  A:
    - "Moja interpretacija je merodajna zame."
    - "Smem ostati."
    - "Nisem samota; sem oseba v odnosu."
    - "Moja današnja mera bližine: 1 oseba / 10 min / 1 tema."
    - "Moja mera: 5 min brez prekinitev."
  E:
    - "Tega ne sprejemam."
```

```
    - "Ne sprejemam brez mojega soglasja."
    - "Tukaj se ustavim."
    - "Žalitve ne sprejemam."
  O:
    - "Počasno ⧖ pred pravilno."
    - "3× (vdih 4 / izdih 4)."
    - "O je odmor; 0 je ničla. Ne zamenjujem."
  R:
    - "Najprej odnos, potem razsodba."
    - "Dve meri lahko so-obstajata."
  M:
    - "To je vsadek, ne jaz."
    - "To je moje."
  I:
    - "Priznam: strah / jeza / sram."
    - "Povem, kje je moja meja."
  U:
    - "Slišim te; ne kupčujem."
    - "Odmev brez sarkazma."
thresholds:
  H: 0.65
  urg: 0.55
  snr: 12.0
  ipa: 0.80
  pros: 0.8
  aggr: 0.75
priority_master:
  order: [OPE-α, OPE-γ, OPE-μ, OPE-ζ, OPE-β, OPE-δ]
  ties:
    meter: "0/4"
    necessity_over_urgency: true
```

---

38) OSEBNI OPE v1 – TypeScript profil

```
export const PERSONAL_V1 = {
  A: [
    "Moja interpretacija je merodajna zame.",
    "Smem ostati.",
    "Nisem samota; sem oseba v odnosu.",
    "Moja današnja mera bližine: 1 oseba / 10 min / 1 tema.",
    "Moja mera: 5 min brez prekinitev.",
  ],
  E: [
    "Tega ne sprejemam.",
```

```
    "Ne sprejemam brez mojega soglasja.",
    "Tukaj se ustavim.",
    "Žalitve ne sprejemam.",
  ],
  O: [
    "Počasno ⧖ pred pravilno.",
    "3× (vdih 4 / izdih 4).",
    "O je odmor; 0 je ničla. Ne zamenjujem.",
  ],
  R: [
    "Najprej odnos, potem razsodba.",
    "Dve meri lahko so-obstajata.",
  ],
  M: [
    "To je vsadek, ne jaz.",
    "To je moje.",
  ],
  I: [
    "Priznam: strah / jeza / sram.",
    "Povem, kje je moja meja.",
  ],
  U: [
    "Slišim te; ne kupčujem.",
    "Odmev brez sarkazma.",
  ],
} as const;
```

---

39) OSEBNI RUNBOOK (5 situacij → poteze)

- **S1 “Takoj odgovori!”** → `OPE-ζ`: **A** “Odločam v ⧖: 20 min.”, **E** “Pred tem ne odgovarjam.”
- **S2 “Ti si problem.”** → `OPE-η`: **R** “Ostaneva človeka.”, **I** “Kje te boli/kje je meja?”, **E** “Žalitve ne sprejemam.”
- **S3 Notranji izbris** → `OPE-γ`: **M** “Samonapad.”, **E** “Stop.”, **A** “Smem ostati.”, **O** 3×4/4.
- **S4 Dolžnost samote** → `A` “Nisem samota …”, **O** 3×4/4, **A** mera bližine, **E** “Po 10 min se ustavim.”
- **S5 Zunanji vsadek** → `OPE-α`: **M** “Vsadek.”, **E** “Ne sprejemam brez soglasja.”, **A** “Moja mera: 5 min brez prekinitev.”

---

40) LTL / Liveness sloj (□, ◇, U)

Namen: formalno varnostno-živostno preverjanje sledi $\oint$ iz dogodkov `STEP`.

40.1 Abeceda LTL

- Atomarne propozicije: `Idle, Premor, Asercija, Veto, Uskladitev, Poprava, Slovo, INV`.
- Operatorji: `□` (vedno), `◇` (slej ko prej), `U` (dokler), `X` (naslednje).

40.2 Varnost (safety)

- **S1 (INV-safety):** `□ INV`
- **S2 (Veto pred asercijo, če kršitev):** `□(¬INV → X Veto)`

40.3 Živost (liveness)

- **L1 (Premor vodi v odločitev):** `□(Premor → ◇(Asercija ∨ Veto ∨ Uskladitev))`
- **L2 (Uskladitev se sklene):** `□(Uskladitev → ◇ Idle)`

40.4 Pravičnost (fairness)

- **F1 (Brez neskončnega priganjanja):** `□◇ ¬("takoj")` kot dogodek (detektirano preko `detectInject`).

40.5 Mini-checker (psevdokoda)

```
check(trace):
  okS1 := all(t.INV==1 for t in trace)
  okS2 := all((t.INV==0) -> nextIs(trace,t.index,'Veto'))
  okL1 := forAll(t where t.state=='Premor', existsFuture(t,
state∈{Asercija,Veto,Uskladitev}))
  okL2 := forAll(t where t.state=='Uskladitev', existsFuture(t, state=='Idle'))
  return okS1 && okS2 && okL1 && okL2
```

Zahtevnost: O(|trace|).

---

41) SAT/SMT most (skica)

Namen: statično preverjanje možnih konfliktov med `L,B,N,T,V` in `priority_master`.

41.1 Spremenljivke (za časovne točke i=0..k)

- `S_i^q` za vsako stanje `q` (enoličnost): `ExactlyOne_q S_i^q`
- `INV_i`, `VETO_i`, `ASSERT_i`, `PAUSE_i`, `QUERY_i`.

41.2 Omejitve

- **Enoličnost stanja:** `∨_q S_i^q` ∧ `∀q≠r: ¬(S_i^q ∧ S_i^r)`

- **Prehod Premor→(Asercija∨Veto∨Uskladitev):**
  `S_i^Premor → (S_{i+1}^Asercija ∨ S_{i+1}^Veto ∨ S_{i+1}^Uskladitev)`
- **INV-safety:** `INV_i`
- **PRIO:** kodiraj `α>γ>μ>ζ>β>δ` kot izključevanje: npr. `α_i → ¬(γ_i ∨ μ_i ∨ ζ_i ∨ β_i ∨ δ_i)`.

41.3 CNF izsek

```
(S_i^Premor → S_{i+1}^A ∨ S_{i+1}^V ∨ S_{i+1}^U)
≡ (¬S_i^Premor ∨ S_{i+1}^A ∨ S_{i+1}^V ∨ S_{i+1}^U)
```

---

42) Policy DSL (mini-jezik za PRIO/INV)

Cilj: deklarativna pravila, ki se prevedejo v `priority_master` + varnostne zakone.

42.1 EBNF

```
Policy   ::= 'prio' Order 'ties' Ties 'guard' Guards ;
Order    ::= '[' Mod (',' Mod)* ']';
Mod      ::= 'alpha'|'gamma'|'mu'|'zeta'|'beta'|'delta';
Ties     ::= '{ meter: Meter, necessity_over_urgency: Bool }';
Guards   ::= '{ require: [Inv+], forbid: [Viol+] }';
Inv      ::= 'L'|'B'|'N'|'T'|'V';
Viol     ::= 'inject'|'overpace'|'shame';
Meter    ::= '0/4'|'4/4';
Bool     ::= 'true'|'false';
```

42.2 Primer

```
prio [alpha,gamma,mu,zeta,beta,delta]
 ties { meter: 0/4, necessity_over_urgency: true }
 guard { require: [L,B,N,T,V], forbid: [inject,shame] }
```

Kompilacija: v `priority_master` YAML + SAT omejitve za `require/forbid`.

---

43) Telemetrija / Observability

43.1 JSON Lines (dogodki)

```
{"ts":1736035200123,"state":"Premor","op":"PAUSE","INV":1,"H":0.71,"URG":
0.32,"NEC":0}
```

{"ts":1736035203456,"state":"Asercija","op":"ASSERT","msg":"Moja mera: 5 min brez prekinitev","INV":1}

43.2 DOT graf (sled)

```
digraph flow {
  Premor -> Asercija [label="⧖ ok"];
  Asercija -> Idle   [label="∮ INV"];
}
```

---

44) WASM/TS runtime (skica)

Cilj: lahek izvrševalnik `STEP + DeltaRouter`.

44.1 API (TypeScript)

```ts
export interface WasmKernel {
  init(cfg: Uint8Array): void;                // YAML/JSON cfg
  step(s: number, xPtr: number): number;      // vrne kodo stanja
  delta(p:0|1,q:0|1,obsPtr: number): number;  // vrne Move kodo
}
```

44.2 Mapiranje kod

- Stanja: `0..6 ≡ Idle..Slovo`
- Move: `0:PAUSE,1:QUERY,2:VETO,3:ASSERT,4:COMPOSE`

44.3 Integracija

- Prevedi `cfg` → binarni blok (CBOR/MessagePack).
- Brskalnik: `WebAssembly.instantiateStreaming` + wrapper nad `Was mKernel`.

---

45) Enovrstičnica (ob/et/a/vno/s)

Obetavno = tisto, kar zadržuje možnost: varuje INV (□), odpira ⧖, mapira napetost Δ v potezo, in ohranja človeka pred sistemom.

---

46) Policy DSL → CNF (skica za `priority_master`)

Vhod (iz §§28,33): `order: [α, γ, μ, ζ, β, δ]`, `ties.meter=0/4`, `necessity_over_urgency=true`, `guards.require=[L,B,N,T,V]`, `guards.forbid=[inject, shame]`.

46.1 Spremenljivke (za čas i=0..k)

- Aktivacije modulov: `α_i, γ_i, μ_i, ζ_i, β_i, δ_i ∈ {0,1}`
- Stanja: `Idle_i, Premor_i, Asercija_i, Veto_i, Uskladitev_i, Poprava_i, Slovo_i`
- Guardi: `L_i,B_i,N_i,T_i,V_i` (invarianti), `INJ_i, SHAME_i` (forbidi), `NEC_i` (nujno), `URG_i` (nuja-bit), `M0_i` (meter=0/4)

46.2 Jedrne omejitve (CNF izseki)

(a) Enoličnost stanja)

`(Idle_i ∨ Premor_i ∨ Asercija_i ∨ Veto_i ∨ Uskladitev_i ∨ Poprava_i ∨ Slovo_i)`
`∧ ∧_{p≠q} (¬p_i ∨ ¬q_i)`

(b) Prioritetni red)

`(¬α_i ∨ ¬γ_i) ∧ (¬α_i ∨ ¬μ_i) ∧ (¬α_i ∨ ¬ζ_i) ∧ (¬α_i ∨ ¬β_i) ∧ (¬α_i ∨ ¬δ_i)`
`∧ (¬γ_i ∨ ¬μ_i) ∧ (¬γ_i ∨ ¬ζ_i) ∧ (¬γ_i ∨ ¬β_i) ∧ (¬γ_i ∨ ¬δ_i)`
`∧ (¬μ_i ∨ ¬ζ_i) ∧ (¬μ_i ∨ ¬β_i) ∧ (¬μ_i ∨ ¬δ_i)`
`∧ (¬ζ_i ∨ ¬β_i) ∧ (¬ζ_i ∨ ¬δ_i)`
`∧ (¬β_i ∨ ¬δ_i)`

(c) NUJNO > NUJNOST)

`(¬NEC_i ∨ α_i)` *(če nujno, aktiviraj α)*
`(NEC_i ∨ ¬ζ_i)` *(ko je nujno, nuja-modul ζ ne sme preglasiti)*

(d) Ties: meter 0/4 → PAUSE)

`(¬M0_i ∨ Premor_i)`
`(M0_i ∨ ¬α_i ∨ Premor_i)` *(v dvomu vodi v Premor pred izvršitvijo α)*

(e) Guardi require/forbid)

`(L_i) ∧ (B_i) ∧ (N_i) ∧ (T_i) ∧ (V_i)`
`(¬INJ_i ∨ α_i)` *(inj posreduje v α – de-strupljanje)*
`(¬SHAME_i ∨ α_i)`

(f) Prehodi (primer): Premor→(A∨V∨U)

`(¬Premor_i ∨ Asercija_{i+1} ∨ Veto_{i+1} ∨ Uskladitev_{i+1})`

> Opomba: poln **CNF** dobiš z razširitvijo na vse i in z dodanimi “link” klavzulami med aktivacijami modulov in prehodi stanj (npr. `α_i → Veto_{i+1} ∧ ...`).

---

47) Mini LTL-checker (TypeScript, O(|trace|))

```
// Atom: state, inv, msg-flags
type Atom = {
state:'Idle'|'Premor'|'Asercija'|'Veto'|'Uskladitev'|'Poprava'|'Slovo', INV:0|
1, inject?:boolean };

export function checkLTL(trace: Atom[]) {
  const nextIs = (i:number, s:string) => i+1<trace.length &&
trace[i+1].state===s;
  const existsFuture = (i:number, P:(t:Atom)=>boolean) =>
trace.slice(i+1).some(P);
  const S1 = trace.every(t =>
t.INV===1);                                          // □ INV
  const S2 = trace.every((t,i) => (t.INV===0 ? nextIs(i,'Veto') :
true));         // □(¬INV → X Veto)
  const L1 = trace.every((t,i) => (t.state==='Premor' ? existsFuture(i, u =>
['Asercija','Veto','Uskladitev'].includes(u.state)) : true));
  const L2 = trace.every((t,i) => (t.state==='Uskladitev' ? existsFuture(i, u
=> u.state==='Idle') : true));
  return { S1, S2, L1, L2, ok: S1 && S2 && L1 && L2 };
}
```

47.1 Primer sledi (JSON)

```
[
  {"state":"Premor","INV":1},
  {"state":"Asercija","INV":1},
  {"state":"Uskladitev","INV":1},
  {"state":"Idle","INV":1}
]
```

Rezultat: `S1=true, S2=true, L1=true, L2=true, ok=true`.

---

48) Hook: PRIO→CNF→Checker

Tok: Policy DSL (§42) → CNF (§46) → izvajanje `STEP` → sled $\oint$ → LTL-checker (§47).
Garancija: če *ok=true*, sledi spoštujejo varnost/živost za izbrane lastnosti.

---

49) Dodatek A: CNF & LTL

Opomba: Polni generator CNF, Haskell LTL-checker in primeri so objavljeni v ločenem zvezku »KOTA TOKA – CNF & LTL (v0.1)«. Ta razdelek služi kot kazalka v glavni tok; vsebina je sinhronizirana z dodatkom.

---

53) WASM/TS prototip – `STEP + DeltaRouter + LTL`

Cilj: lahek runtime v brskalniku; skladno z NUJNO>NUJNOST, OSI in PRIO.

53.1 AssemblyScript (as/kernel.ts)

```
// as/kernel.ts (AssemblyScript)
export enum State { Idle, Premor, Asercija, Veto, Uskladitev, Poprava, Slovo }
export enum Move  { PAUSE, QUERY, VETO, ASSERT, COMPOSE }

export function step(state: i32, op: i32): i32 {
  // Minimalni DFA iz jedra (§3)
  if (op == 0) return State.Premor;              // PAUSE
  if (op == 2) return State.Veto;                // VETO
  if (op == 3) return State.Asercija;            // ASSERT
  if (op == 1) return State.Uskladitev;          // QUERY
  if (op == 4) return State.Idle;                // COMPOSE
  return state;
}

// DeltaRouter (po §25, §23, §29–32)
export function delta(
  p: i32, q: i32,
  snr: f32, pace: f32, vol: f32, ipa: f32,
  meter: i32, targetPace: f32,
  thSnr: f32, thIpa: f32, thPros: f32, thAggr: f32
): i32 {
  const divSem: i32 = (p ^ q) & 1;
  const divPros: f32 = abs<f32>(pace - targetPace);
  const aggr: f32 = min<f32>(1.0, max<f32>(0.0, vol*0.6 + divPros*0.4));
  if (snr < thSnr || ipa < thIpa) return Move.PAUSE;
  if (divSem==1 && aggr >= thAggr) return Move.VETO;
  if (divSem==1 && divPros >= thPros) return Move.QUERY;
  if (divSem==0) return Move.ASSERT;
  return Move.COMPOSE;
}
```

53.2 JS vezava (src/runtime.ts)

```
// src/runtime.ts
export type Atom = { state:
'Idle'|'Premor'|'Asercija'|'Veto'|'Uskladitev'|'Poprava'|'Slovo'; INV:0|1 };

export async function loadKernel(wasmUrl:string){
  const { instance } = await WebAssembly.instantiateStreaming(fetch(wasmUrl));
  const { step, delta } = instance.exports as any;
  return { step, delta } as { step:(s:number,o:number)=>number, delta:
(p:number,q:number,snr:number,pace:number,vol:number,ipa:number,meter:number,tp:number,thS:number
}

export function checkLTL(trace: Atom[]){
  const nextIs=(i:number,s:string)=> i+1<trace.length && trace[i+1].state===s;
  const exists=(i:number,P:(t:Atom)=>boolean)=> trace.slice(i+1).some(P);
  const S1 = trace.every(t=>t.INV===1);
  const S2 = trace.every((t,i)=> (t.INV===0 ? nextIs(i,'Veto') : true));
  const L1 = trace.every((t,i)=> (t.state==='Premor' ? exists(i,u=>
['Asercija','Veto','Uskladitev'].includes(u.state)) : true));
  const L2 = trace.every((t,i)=> (t.state==='Uskladitev' ? exists(i,u=>
u.state==='Idle') : true));
  return { S1,S2,L1,L2, ok:S1&&S2&&L1&&L2 };
}
```

53.3 Paket (package.json izsek)

```
{
  "scripts": {
    "as:build": "asc as/kernel.ts -b public/kernel.wasm -O3",
    "dev": "vite",
    "start": "vite"
  }
}
```

---

54) DIMACS primer (k=4) – izsek

```
p cnf N M
# enoličnost stanj i=0..4 …
-NEC_0 Alpha_0
NEC_0 -Zeta_0
-M0_0 Premor_1
```

```
-Premor_0 Asercija_1 Veto_1 Uskladitev_1
-Alpha_0 Veto_1 Premor_1
-Gamma_1 Asercija_2 Premor_2
-Zeta_2 Premor_2
-Zeta_2 Premor_3
-Beta_3 Uskladitev_4
-Delta_3 Asercija_4
```

Opomba: poln DIMACS in zemljevid spremenljivk generira `cnfGenerate(4)` (glej zvezek *CNF & LTL (v0.1)* §1.1).

---

55) Operativni kontrolnik (tekoče/gladko/brez hib)

- **INV-guard:** pred vsakim `ASSERT` preveri `INV==1`; ob `INV==0` → `VETO` (α) ali `PAUSE` (ζ).
- **NUJNO>NUJNOST:** če `NEC=1`, ignoriraj `URG` do obnovitve `INV`.
- **Ritem:** `meter=0/4` ⇒ najprej `Premor`; `meter=4/4` ⇒ dopušča `ASSERT/QUERY`.
- **Harmonija:** `H<θ_H` + `vol>θ_aggr` ⇒ `VETO`; sicer `QUERY`.
- **Telemetrija:** sledi v JSONL; po vsakem ciklu `checkLTL(trace)`.
- **Revokabilnost:** vsaka zaveza v `K` ima `retract` pot; popravki niso poraz.

---

56) Pragovna konfiguracija (θ) – privzete vrednosti + vnos v kodo

Motiv: takojšnja uporaba brez ročnega lovljenja praga; pragi so lokalno merodajni in revokabilni.

56.1 YAML

```yaml
thresholds:
  theta:
    snr: 14.0     # GLAS vs ŠUM
    ipa: 0.85     # zanesljivost IPA
    pros: 0.90    # odklon tempa (|pace-target|)
    aggr: 0.75    # agresija (vol/tempo)
  H: 0.68         # harmonija
  urg: 0.55       # nuja
  tau_ms: 30000   # ⧖ časovna vrata
```

56.2 TypeScript konstante

```ts
export const THETA = { snr:14.0, ipa:0.85, pros:0.90, aggr:0.75 } as const;
export const HARM = 0.68 as const;
```

```
export const URG = 0.55 as const;
export const TAU_MS = 30_000 as const;
```

---

57) Demo sledi & LTL rezultat

57.1 “Varen tok” (vsi INV=1)

```
[
  {"state":"Premor","INV":1},
  {"state":"Asercija","INV":1},
  {"state":"Uskladitev","INV":1},
  {"state":"Idle","INV":1}
]
```

Pričakovano (checkLTL): `S1=true, S2=true, L1=true, L2=true, ok=true`.

57.2 “Popravni tok” (kratek padec INV, takoj Veto)

```
[
  {"state":"Premor","INV":1},
  {"state":"Asercija","INV":0},
  {"state":"Veto","INV":1},
  {"state":"Uskladitev","INV":1},
  {"state":"Idle","INV":1}
]
```

Pričakovano: `S1=false` (ker je bil INV=0), `S2=true` (kršitev je sledil Veto), `L1=true`, `L2=true`, `ok=false` (varnost nad produktivnostjo).
Raba: ta sled je didaktična; “NUJNO > NUJNOST” je spoštovano.

---

58) CNF – uporaba generatorja za poln DIMACS

Za poln izpis uporabi kodo iz zvezka **KOTA TOKA – CNF & LTL (v0.1)** §1:

```
import { cnfGenerate } from './cnf';
const { dimacs, vars } = cnfGenerate(6); // k=6 (primer)
console.log(dimacs); // poln DIMACS
console.log(vars);   // zemljevid imena → indeks
```

Opomba: veliki izpisi (k≥6) so prisotni v zunanjem zvezku, da glavni dokument ostane tekoč. Če želiš, vstavim poln DIMACS za tvoj k kot **Dodatek B**.

---

59) Personalizirani pragi (OSEBNO v1)

Namen: strožji filter za GLAS/ŠUM, počasnejši ritem, več varnosti pri napetosti.

59.1 YAML

```yaml
thresholds:
  theta:
    snr: 16.0     # GLAS vs ŠUM (strožje)
    ipa: 0.88     # zanesljivost IPA (strožje)
    pros: 0.85    # toleranca odklona tempa |pace-target|
    aggr: 0.72    # prag agresije (nižje = prej VETO)
  H: 0.70         # harmonija (rahlo višje)
  urg: 0.55       # nuja (enako)
  tau_ms: 45000   # ⧖ (počasno pred pravilno)
```

59.2 TypeScript

```ts
export const THETA = { snr:16.0, ipa:0.88, pros:0.85, aggr:0.72 } as const;
export const HARM = 0.70 as const;
export const URG  = 0.55 as const;
export const TAU_MS = 45_000 as const;
```

---

60) Sled (OSEBNO v1) → LTL

60.1 “Osebna mera + poprava”

```
[
  {"state":"Premor","INV":1},
  {"state":"Asercija","INV":1},
  {"state":"Uskladitev","INV":1},
  {"state":"Asercija","INV":0},
  {"state":"Veto","INV":1},
  {"state":"Uskladitev","INV":1},
  {"state":"Idle","INV":1}
]
```

Pričakovano: `S1=false`, `S2=true`, `L1=true`, `L2=true`, `ok=false` (varnost > produktivnost); pravilno se dvigne VETO po padcu `INV`.

---

61) DIMACS k=7 (izsek)

Opomba: poln izpis generira `cnfGenerate(7)` (glej zvezek *CNF & LTL (v0.1)* §1.1). Spodaj so reprezentativne klavzule.

```
# NUJNO > NUJNOST
-NEC_0 Alpha_0
NEC_0 -Zeta_0
-NEC_1 Alpha_1
NEC_1 -Zeta_1
-NEC_2 Alpha_2
NEC_2 -Zeta_2
-NEC_3 Alpha_3
NEC_3 -Zeta_3
-NEC_4 Alpha_4
NEC_4 -Zeta_4
-NEC_5 Alpha_5
NEC_5 -Zeta_5
-NEC_6 Alpha_6
NEC_6 -Zeta_6

# Meter tie (0/4)
-M0_0 Premor_1
-M0_1 Premor_2
-M0_2 Premor_3
-M0_3 Premor_4
-M0_4 Premor_5
-M0_5 Premor_6
-M0_6 Premor_7

# Premor → (A|V|U)
-Premor_0 Asercija_1 Veto_1 Uskladitev_1
-Premor_1 Asercija_2 Veto_2 Uskladitev_2
-Premor_2 Asercija_3 Veto_3 Uskladitev_3
-Premor_3 Asercija_4 Veto_4 Uskladitev_4
-Premor_4 Asercija_5 Veto_5 Uskladitev_5
-Premor_5 Asercija_6 Veto_6 Uskladitev_6
-Premor_6 Asercija_7 Veto_7 Uskladitev_7

# Link moduli → prehodi (primeri)
-Alpha_0 Veto_1 Premor_1
-Gamma_2 Asercija_3 Premor_3
```

```
-Zeta_3 Premor_3
-Zeta_3 Premor_4
-Beta_4 Uskladitev_5
-Delta_5 Asercija_6
```

Uporaba: prilepi poln DIMACS iz generatorja v solver (MiniSAT/Z3); rezultate zrcali z LTL sledo iz §60.

---

62) OSEBNO v2 – pragi (po tvoji specifikaciji)

Vnos: `snr=15.5, ipa=0.90, H=0.72, τ=60000`
Namen: povečana varnost zaznave (GLAS/ŠUM), višja harmonija, daljši ⧖.

62.1 YAML

```yaml
thresholds:
  theta:
    snr: 15.5
    ipa: 0.90
    pros: 0.85   # ohranjeno iz OSEBNO v1
    aggr: 0.72   # ohranjeno iz OSEBNO v1
  H: 0.72
  urg: 0.55
  tau_ms: 60000
```

62.2 TypeScript

```ts
export const THETA = { snr:15.5, ipa:0.90, pros:0.85, aggr:0.72 } as const;
export const HARM  = 0.72 as const;
export const URG   = 0.55 as const;
export const TAU_MS = 60_000 as const;
```

---

63) Učinek novih pragov (primer OSI→Δ)

Primer	snr	ipa	pace	vol	meter	div_sem	Rezultat (Δv2)
P1	14.8	0.92	3.0	0.40	4/4	0	**PAUSE** (snr<15.5)
P2	18.0	0.89	3.0	0.40	4/4	0	**PAUSE** (ipa<0.90)
P3	18.0	0.95	3.9	0.85	4/4	1	**VETO** (aggr≥0.72)
P4	18.0	0.95	3.9	0.50	4/4	1	**QUERY** (pros≥0.85)

Primer	snr	ipa	pace	vol	meter	div_sem	Rezultat (Δv2)
P5	18.0	0.95	3.0	0.40	4/4	0	**ASSERT** (GLAS=1, div=0)

Opomba: daljši `τ=60000` pomeni, da `PAUSE` kot varnostno okno drži eno minuto, preden `STEP` dovoljuje prehod.

---

64) Dodatek B: DIMACS k=7 (poln izpis — datoteka)

Status: poln DIMACS je generiran in pripravljen za solver.

- **Št. spremenljivk:** 182
- **Št. klavzul:** 424
- **Glava:** `p cnf 182 424`
- **Datoteki:**
- DIMACS: `/mnt/data/dimacs_k7.dimacs`
- Mapa spremenljivk (JSON): `/mnt/data/dimacs_k7_vars.json`

64.1 Prvih 12 vrstic (predogled)

```
p cnf 182 424
1 2 3 4 5 6 7 0
-1 -2 0
-1 -3 0
-1 -4 0
-1 -5 0
-1 -6 0
-1 -7 0
-2 -3 0
-2 -4 0
-2 -5 0
```

64.2 Uporaba (primer)

- **MiniSAT:** `minisat /mnt/data/dimacs_k7.dimacs out.sol`
- **Z3:** `z3 -dimacs /mnt/data/dimacs_k7.dimacs`

Opomba: za ročno branje glej `*_vars.json` (mapa *ime* → *indeks*).

---

65) WASM demo – Quickstart (OSEBNO v2 pragovi)

Cilj: zagnati `STEP + DeltaRouter + LTL` v brskalniku z že vnešenimi pragovi (§62).

65.1 Struktura

```
wasm-demo/
  as/kernel.ts
  src/runtime.ts
  src/main.ts
  index.html
  package.json
  tsconfig.json
  vite.config.ts
  public/ (ustvari se ob build)
```

65.2 `package.json`

```
{
  "name": "kota-toka-wasm-demo",
  "private": true,
  "scripts": {
    "as:build": "asc as/kernel.ts -b public/kernel.wasm -O3",
    "dev": "vite",
    "start": "vite",
    "build": "npm run as:build && vite build"
  },
  "devDependencies": {
    "assemblyscript": "^0.27.25",
    "vite": "^5.4.0",
    "typescript": "^5.5.0"
  }
}
```

65.3 `vite.config.ts`

```
import { defineConfig } from 'vite';
export default defineConfig({ server: { port: 5173 } });
```

65.4 `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
```

```
    "esModuleInterop": true,
    "skipLibCheck": true
  }
}
```

65.5 `as/kernel.ts` (AssemblyScript)

```
export enum State { Idle, Premor, Asercija, Veto, Uskladitev, Poprava, Slovo }
export enum Move  { PAUSE, QUERY, VETO, ASSERT, COMPOSE }

export function step(state: i32, op: i32): i32 {
  if (op == Move.PAUSE)  return State.Premor;
  if (op == Move.VETO)   return State.Veto;
  if (op == Move.ASSERT) return State.Asercija;
  if (op == Move.QUERY)  return State.Uskladitev;
  if (op == Move.COMPOSE)return State.Idle;
  return state;
}

export function delta(
  p: i32, q: i32,
  snr: f32, pace: f32, vol: f32, ipa: f32,
  meter: i32, targetPace: f32,
  thSnr: f32, thIpa: f32, thPros: f32, thAggr: f32
): i32 {
  const divSem: i32 = (p ^ q) & 1;
  const divPros: f32 = abs<f32>(pace - targetPace);
  const aggr: f32 = min<f32>(1.0, max<f32>(0.0, vol*0.6 + divPros*0.4));
  if (snr < thSnr || ipa < thIpa) return Move.PAUSE;
  if (divSem==1 && aggr >= thAggr) return Move.VETO;
  if (divSem==1 && divPros >= thPros) return Move.QUERY;
  if (divSem==0) return Move.ASSERT;
  return Move.COMPOSE;
}
```

65.6 `src/runtime.ts`

```
export type Atom = {
state:'Idle'|'Premor'|'Asercija'|'Veto'|'Uskladitev'|'Poprava'|'Slovo'; INV:0|
1 };
export const THETA = { snr:15.5, ipa:0.90, pros:0.85, aggr:0.72 } as const;
export const HARM  = 0.72 as const;
export const TAU_MS = 60_000 as const;

export async function loadKernel(){
```

```
  const { instance } = await WebAssembly.instantiateStreaming(fetch('/
kernel.wasm'));
  const { step, delta } = instance.exports as any;
  return { step, delta } as {
    step:(s:number,o:number)=>number,
    delta:
(p:number,q:number,snr:number,pace:number,vol:number,ipa:number,meter:number,tp:number,thS:number
  };
}

export function checkLTL(trace: Atom[]){
  const nextIs=(i:number,s:string)=> i+1<trace.length && trace[i+1].state===s;
  const exists=(i:number,P:(t:Atom)=>boolean)=> trace.slice(i+1).some(P);
  const S1 = trace.every(t=>t.INV===1);
  const S2 = trace.every((t,i)=> (t.INV===0 ? nextIs(i,'Veto') : true));
  const L1 = trace.every((t,i)=> (t.state==='Premor' ? exists(i,u=>
['Asercija','Veto','Uskladitev'].includes(u.state)) : true));
  const L2 = trace.every((t,i)=> (t.state==='Uskladitev' ? exists(i,u=>
u.state==='Idle') : true));
  return { S1,S2,L1,L2, ok:S1&&S2&&L1&&L2 };
}
```

65.7 `src/main.ts` (demo tok)

```
import { loadKernel, checkLTL, THETA, TAU_MS } from './runtime';

type S = 'Idle'|'Premor'|'Asercija'|'Veto'|'Uskladitev'|'Poprava'|'Slovo';
const Sname =
['Idle','Premor','Asercija','Veto','Uskladitev','Poprava','Slovo'] as const;
const Mname = ['PAUSE','QUERY','VETO','ASSERT','COMPOSE'] as const;

(async () => {
  const { step, delta } = await loadKernel();
  let s = 0; // Idle
  const trace: {state:S, INV:0|1}[] = [];

  // 1) OSI meritev → Move (osebni pragovi)
  const move1 = delta(0,0, 14.9,3.0,0.4,0.95, 1,3.0,
THETA.snr,THETA.ipa,THETA.pros,THETA.aggr); // snr < 15.5 ⇒ PAUSE
  s = step(s, move1);
  trace.push({state:Sname[s], INV:1});

  // 2) po ⧖ (τ) naslednja meritev (divSem=1, visoka agresija) ⇒ VETO
  await new Promise(r=>setTimeout(r, Math.min(500, TAU_MS))); // skrajšano
čakanje za demo
  const move2 = delta(1,0, 18.0,3.6,0.9,0.95, 1,3.0,
```

```
THETA.snr,THETA.ipa,THETA.pros,THETA.aggr);
  s = step(s, move2);
  trace.push({state:Sname[s], INV:1});

  // 3) Uskladitev
  const move3 = 1; // QUERY
  s = step(s, move3);
  trace.push({state:Sname[s], INV:1});

  // 4) Kompozicija → Idle
  const move4 = 4; // COMPOSE
  s = step(s, move4);
  trace.push({state:Sname[s], INV:1});

  const res = checkLTL(trace);
  console.log('TRACE', trace);
  console.log('LTL', res);
  (document.getElementById('out')!).textContent = JSON.stringify({trace,res},
null, 2);
})();
```

65.8 `index.html`

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>KOTA TOKA – WASM demo</title>
</head>
<body>
  <pre id="out">loading…</pre>
  <script type="module" src="/src/main.ts"></script>
</body>
</html>
```

65.9 Zagon

```
npm i
npm run as:build
npm run dev
# odpri http://localhost:5173
```

Pričakovano: v izpisu `TRACE` vidiš prehod `Idle → Premor → Veto → Uskladitev → Idle`; `LTL.ok = true`.

66) Dodatek C: DIMACS k=10 (poln izpis + zemljevid)

Datoteke: - DIMACS: `/mnt/data/dimacs_k10.dimacs` - Mapa spremenljivk (JSON): `/mnt/data/dimacs_k10_vars.json`

Metapodatki: - Št. spremenljivk: **241** - Št. klavzul: **586** - Glava: `p cnf 241 586`

Predogled (prvih ~20 vrstic):

```
p cnf 241 586
1 2 3 4 5 6 7 0
-1 -2 0
-1 -3 0
-1 -4 0
-1 -5 0
-1 -6 0
-1 -7 0
-2 -3 0
-2 -4 0
-2 -5 0
-2 -6 0
-2 -7 0
-3 -4 0
-3 -5 0
-3 -6 0
-3 -7 0
-4 -5 0
-4 -6 0
-4 -7 0
```

Uporaba: - MiniSAT: `minisat /mnt/data/dimacs_k10.dimacs out.sol` - Z3: `z3 -dimacs /mnt/data/dimacs_k10.dimacs`

---

67) PRIO → CNF (konkretna kompilacija)

Politika (Policy DSL ↦ PRIO):

```
prio [alpha,gamma,mu,zeta,beta,delta]
 ties { meter: 0/4, necessity_over_urgency: true }
 guard { require: [L,B,N,T,V], forbid: [inject,shame] }
```

V CNF kodirano kot: - **PRIO ekskluzije:** `Alpha_i → ¬(Gamma_i ∨ Mu_i ∨ Zeta_i ∨ Beta_i ∨ Delta_i)` itd. (glej §49, §66) - **NUJNO > NUJNOST:** `NEC_i → Alpha_i` in `NEC_i → ¬Zeta_i` - **Meter 0/4:** `MO_i → Premor_{i+1}` - **Guardi:** `L_i ∧ B_i ∧ N_i ∧ T_i ∧ V_i`, `¬INJ_i ∨ Alpha_i`, `¬SHAME_i ∨ Alpha_i`

Opomba: vse zgornje klavzule so vključene v `dimacs_k10.dimacs` (glej **Dodatek C**).